Case Study 6: Microsoft Malware Detection

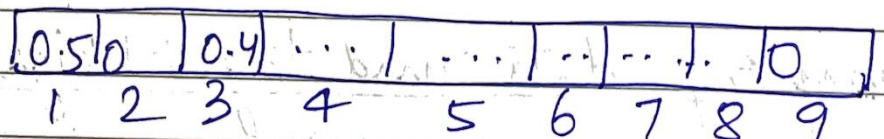
Business/Real World Problem

V1: To classify malware into 9 categories.

V2: Objectives and Constraints

1) Minimize multi-class error.
2) Multi-class probability estimates.
3) Malware detection should not take hours and block the user's computer. It should finish in a few seconds or a minute.

→ Prob. estimates



File can either class 1 or 3 based on near probabilities.

V3: Machine Learning Problem Mapping: Data Overview

.ASM file → Assembly level code file

~~.data~~
.byte ## → Hexadecimal level code for binary data.

V4: Machine Learning problem mapping:

ML problem:

9 class classification problem.

Performance Metric

1) Multi-class log loss.
2) Confusion matrix

1) Multiclass log loss

Let $D = \{x_i, y_i\}_{i=1}^{n}$, c = classes

$$= -\frac{1}{n} \sum_{i=1}^{n} \sum_{j=1}^{c} y_{ij} \log p_{ij}$$

y_{ij} — ground truth

p_{ij} — model predicted value.

$y_{ij} = 1$ if $x_i \in$ class j
else
0

$y_{i,2}$ — class

datapoint

P_{ij} = Probability that x_i belongs to class j.

Case 1: $p_{12} = 1$

$p_{11} = 0$

$p_{13} = 0$

⋮

$p_{19} = 0$

$i = 1, \; j = 2$

$-y_{ij} \log p_{ij}$

$1 \quad 0 \quad \because \log(1) = 0$

$\Downarrow$

0 (log loss)

Case 2: $p_{11} = 0.1$

$p_{12} = 0.9$

$p_{13} = 0$

⋮

$p_{19} = 0$

$-y_{ij} \log p_{ij}$

$\Downarrow \quad \Downarrow$

$-\log(0.9)$

$\Rightarrow 0.045$ which is more than 0. (log loss)

$y_{11} = 0$

$y_{12} = 1$

$y_{13} = 0$

⋮

$P_{11} = 0.1$

Case 3 : $P_{12} = 0.1$

$P_{13} = 0.8$

$\Rightarrow -y_{ij} \log P_{ij}$

$\Rightarrow -1 \times \log(0.1)$

$\Rightarrow$ 1 = log loss (base-10)

Ideal prob. value is 1, log loss = 0.

As P_{ij} deviates from its ideal value, log-loss will increase.

Obj: Predict probability

constraints: 1) class prob are needed.

2) Penalize the errors (LogLoss)

3) Some latency constraints.

V5: Machine Learning problem mapping: Train and Test Splitting

Train } 80 → 64%
Cross Validation } 80 → 16%
Test → 20 → 20%

Randomly splitting of data.

Check out useful blogs, videos & Reference papers.

V6: Exploratory Data Analysis: Class distributions

→ Separation of byte files and asm files into different folders.

→ Multiclass classification of imbalanced data.

V7: EDA: Feature Extraction from byte files

→ Feature engineering on byte file.

os.Stat // See Ref.

→ Boxplot of filesize.

- class2 & 5 overlap but some nonoverlap also.

→ Remove the Starting address.

256 different bytes in the file (00-ff)

→ We can employ Unigrams. bag of words.

→ CountVectorizer doesn't scale to this much size of data.
So developed their own BoW.

→ Normalization.

V8: EDA: Multivariate analysis of features from byte files

→ tsne (plotted with multiple perplexities)

V9: EDA: Train Test class distributions

% of classes in Train & Test data plot.

V10: ML models – using byte files only: Random Model

Multi-class log loss varies b/w

$$[0, \infty)$$

Vector of size 9 and fill random values for log loss and if prob they should ~~some~~ sum to 1 after normalization.

1) generate P_{ij} as random values.
2) S = sum of P_{ij} for all j
3) P_{ij} / S

Largest Multi-class log loss can go upto 2.45.

Thus, using Random Model to check how bad a model could get.

In Recall row sum to 1.

In Precision colum sum to 1.

VII: KNN (On byte files)

along with calibrated classifier.
∵ we need prob. values.

Error vs. k plot on CV

Precision Matrix shows 5th class has very low value showing that the no. of datapoints for that class is very low.

V12: Logistic Regression

In KNN, no. of misclassified points are 4.50% with test log loss of 0.24%.

For LR, Miss-classification rate = 12.39%

LR not working too well for this model.
→ V13

V14: ASM Files: Feature extraction & Multiprocessing

52 keywords in asm file.

We use BoW for them in multithreaded environment.

V15: File-Size feature

Box plot on file-size of .asm files.

Box 2 is biggest, which means, class 2 files are larger.

V16: Univariate Analysis

It is done on filesize and on 52 features.

We do normalization first.

V17: t-SNE Analysis (Multivariate Analysis)

Not clean.

Conclusion. → Only 52 features were all out of which only on some of them were used for univariate analysis.

V13: Random Forest and XGBoost

Only on byte files. Random Forest: It might work well on 25% features

Hyperparameter = n_estimators.

1000 Tree work well. log loss is less.

log loss = 0.085
No. of misclassified points = 2.02%

Xgboost: (500 trees)
n_estimators was used as Hyperparameter
log loss = 0.079

% of misclassified pts = 1.24%

Random Search was also used for Xgboost, since Xgboost could be trained on multiple parameters.

test loss = 0.078

~~% of mi~~

V18: ML Models on ASM file features (better performer)

Train - CV - Test split → 64% / 16 / 20%.

1) KNN → K = 3
log loss = ~~0.088~~ 0.089
% of misclassified pts = 2.02%

2) Logistic Regression
log loss = 0.415
% of misclassified pts = 9.61%
Not good.

3) RF
n_estimators = 500

log loss = 0.057
% of misclassified pts. = 0.15%

4) Xgboost

log loss = 0.0491

Random Search

log loss = 0.048
Misclassification % 0.87%

V19: Models on all features : t-SNE

256+1 (byte) + 52+1 (ASM) ⇒ Concatenation of both file features.

perplexity = 50 } pts are similar.
30

V20: Models on all features : Rf + Xgboost

310 features

RF:

log loss = ~~0.0355~~ 0.040

misclassification % =< 1%

Xgboost: log loss =
Random Search = 0.031

Case Study 6: Microsoft Malware Detection Assignment

Assignment Tasks

1) Add bi-grams and n-grams.
   hexa decimal BoW on byte files.

2) Code on github & log loss <= 0.01

Optional

3) Video on Kaggle Solution, implement the image feature to improve the loglos.